Software Engineering

Lecture Eight

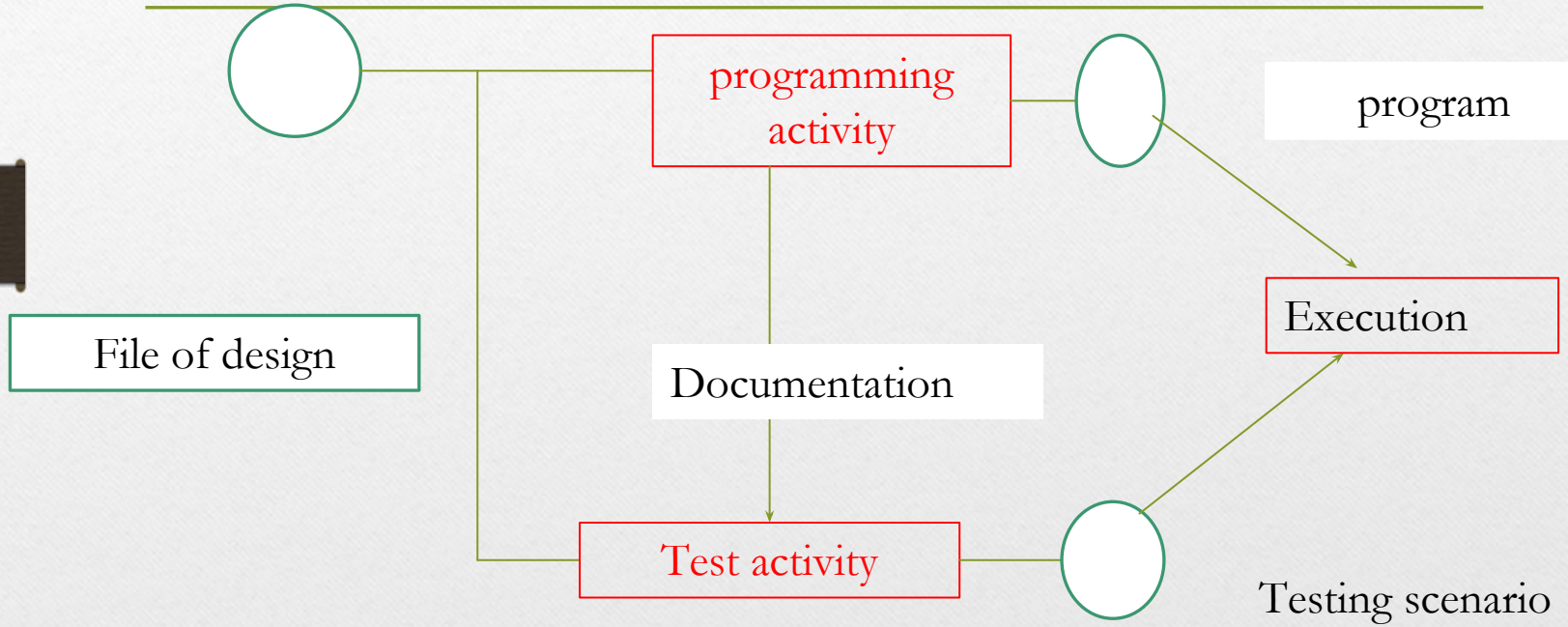
Testing

Testing is the process of executing program in order to find errors. The test can not show the absence of defects, it can only show that the software defects are present.

The test permit to ensure that what we programmed is correct, this mean if we provide the program with the required data, this one will give in the output the wanted results, in the required time. This process consumes the authorized resources and diagnoses an error and the resources of this error.

The activity of test and the activity of programming constitute a form of each of the activities requires a certain effort, but what it is necessary to optimise in the sum.

Ef(total)=Ef(programming)+Ef(Test)



The test concerns program execution, but it is not the only way to correct a program numerous mistakes can be found, regardless of all execution, by means of design review code inspection, proofs, etc. the effort dedicated to discover errors is the sum that it is necessary to optimize:

Ef(error)=Ef(review)+Ef(Test)

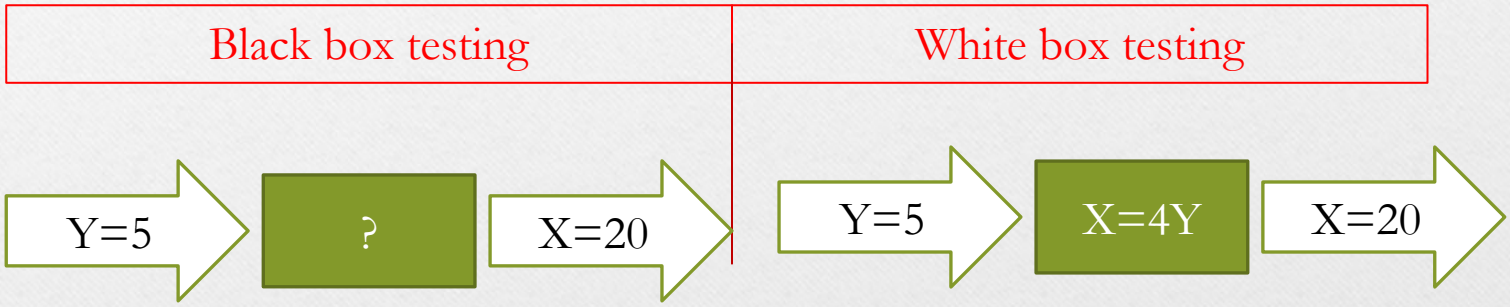
The main objectives of testing are:

1. detect the deviations in relations to the specifications
2. detect errors
3. increase the confidence in the program
4. determine the reliability level in the software
5. evaluate the performance
6. evaluate the behavior undercharge

Testing techniques

There are two different classes of testing techniques for testing software :

1. **White box** test techniques: testing the internal program logic.
2. **Black box** test techniques: testing of software requirements.



White box testing techniques

White box testing is a test case design that derives test cases using the control structure of the procedural design.

Using White box testing methods, the software engineer can derive test cases that:

1. Guarantee that all independent paths are exercised at least once.
2. Test the possibilities (“true” and ”false”)of all logical decisions.
3. Execute all loop with their different values (at their boundaries and within operational bounds)
4. Test the internal data structures to ensure validity.

The white box testing permits to regroup errors in the following categories:

a) The logic errors and the incorrect specifications.

b) Boolean errors.

c) Errors of repetitive instructions.

d) Errors of data type.

e) Typographic (syntax) and random errors.

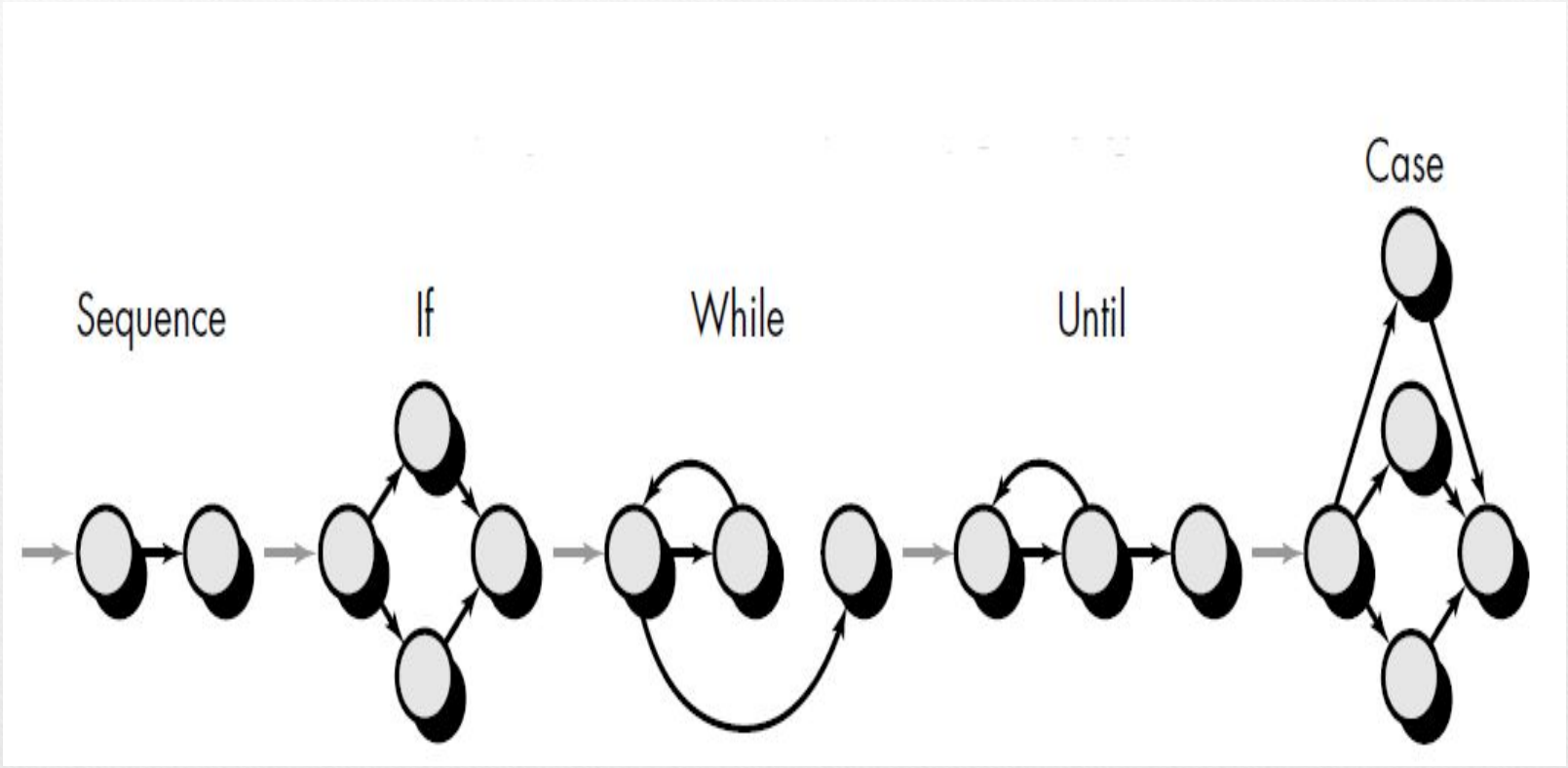
The white box testing techniques are:

Basic path testing

this method enables the designer to derive measures of a procedural design and uses this measure as a guide to define a set of execution **paths**. The basis path method permits to execute every statement in the program at least once. This method can be applied to source code. The basis path test techniques transforms the program into ***control graphs*** in order to deduce the ***measure of complexity***

Control graph

The control graph of a program is the graph that represents the set of paths corresponding to program execution. The instruction is represented as follows:



According to the symbols each circle, called a control or flow graph ***node***, represents one or more instructions. The arrows on the flow graph, called ***edges***, represent flow of control. The zones limited by edges and nodes are called ***regions***. When counting the number of regions we count the zone outside the graph as a region. The node that represent a condition is called ***predicate node*** and it has two or more edges emanating from it.

Measure of complexity:

The measure is also called cyclomatic number or cyclomatic complexity. It gives the number of minimal paths. The cyclomatic number can be calculated by the following formulas:

Cyclomatic number:

No. of edges- No. of nodes +2

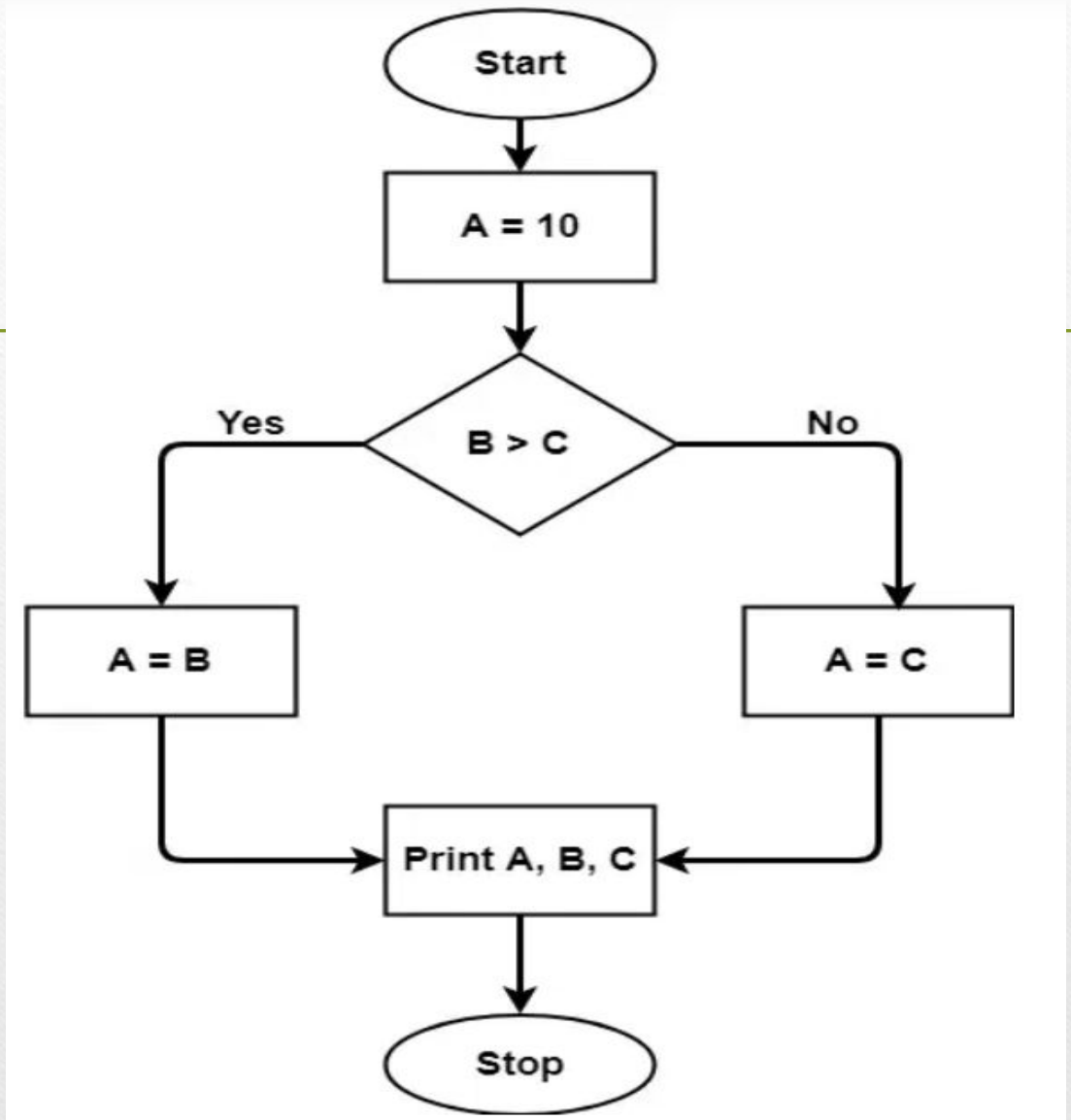
No. Of regions

No. Of conditions+1

The resulting value of the cyclomatic number determines the number of independent paths and indicates the maximal number of tests which must be executed to ensure that all statement have been executed at least once.

Examples

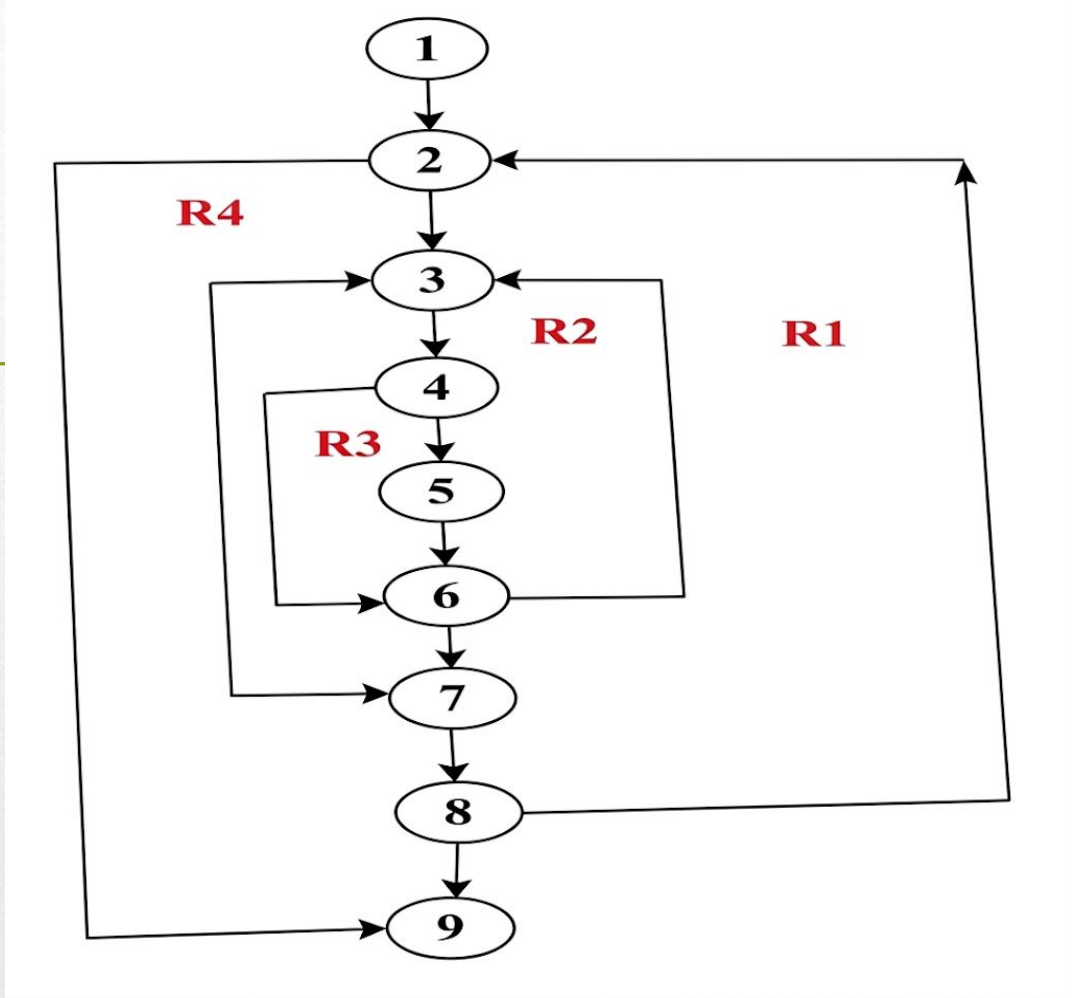
- A = 10
  IF B > C THEN
  A = B
  ELSE
  A = C
  ENDIF
  Print A
  Print B
  Print C


Start
A = 10
B > C
Yes
No
A = B
A = C
Print A, B, C
Stop

Draw the control graph and find the Cyclomatic number of the following program:

```
1)void selectionsort(int[] table) {
        int   min, pos, temp;
 2)   for (int  i=0;i<table.length-1;i++)
        { min=table[i];
3)          for (j=i+1 ;j<table.length ;j++)
4)               if (table[j]<min)
5)                 {
                   min=table[j];
                     pos=j;
6)                {
7)               temp=table[i];
                 table[i]=min;
                 table[pos]=temp;
8)                }}//  end for
9)         }//end selectionsort
```

Flow graph:
(there are 4 regions :R1,R2,R3, and R4)



Cyclomatic complexity:

Cyclomatic complexity=number of regions in the flow graph =4
Or: number of condition nodes+1=3+1=4
Or: number of edges-number of nodes+2=11-9+2=4